SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Joanathan Packia Singh	**Reg. No.:**	192472229
Date:	9/8/26	**Duration:**	60 minutes

Code of Conduct

I Joanathan Packia Singh/192472229 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. **Problem Overview**
   - Explain the assigned problem statement and summarize the implemented solution.
   - Describe the project objectives and key functionalities.
2. **Repository Organization**
   - Evaluate the repository structure, folder organization, and file naming conventions.
   - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review**
   - Review the source code for readability, modularity, coding standards, and documentation.
   - Identify strengths and areas for improvement.
4. **Version Control Evaluation**
   - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
   - Explain how version control supports collaborative software development.
5. **Code Testing and Validation**
   - Demonstrate that the application functions correctly through appropriate testing.
   - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**
   - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

- Suggest improvements to enhance usability and maintainability.

7. **Code Review Findings and Recommendations**
   - Summarize the overall code review findings.
   - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15%)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

	complete, clear, and well organized.			
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

SIMATS ENGINEERING

(Saveetha School of Engineering)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 — CODE REVIEW & REPOSITORY EVALUATION

Fitness and Diet Tracker with Personalized Recommendations

Course Code	CSA10 / CSA1063
Course Title	Software Engineering
CO Assessed	CO3 — Build full-stack applications with an emphasis on containerization, version control, and collaborative development
Assessment	Assessment Tool 2 — Code Review & Repository Evaluation
Weightage / Total Marks	20% / 25 Marks
Student Name	Joanathan Packia Singh
Register Number	192472229
Assigned Problem No.	Problem 22 — Fitness and Diet Tracker with Personalized Recommendations
Repository Reviewed	FitTrack Pro — github.com/JoanathanPS/fittrack-pro
Duration	60 minutes

Code of Conduct

I, Joanathan Packia Singh (Reg. No. 192472229), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

TABLE OF CONTENTS

1. Problem Overview

1.1 Assigned Problem Statement

The assigned problem (Problem 22) calls for a Fitness and Diet Tracker with Personalized Recommendations — a platform that lets users monitor daily physical activity, calorie intake, and nutritional habits, then converts that data into personalized workout and diet guidance. The implemented solution, FitTrack Pro, addresses this through a five-service architecture: a React client, a Node.js/Express API, a Python recommendation microservice, MongoDB for persistence, and Redis for caching and reminders.

1.2 Summary of the Implemented Solution

The repository under review implements the core backend skeleton of FitTrack Pro: a BMI calculation controller, a base Express router that exposes health and BMI endpoints, centralized error-handling middleware, and a minimal React entry component for the client. The codebase is small but functional — it runs, its single unit test suite passes, and its Git history shows a complete Gitflow cycle (feature branches, an integration merge with a real conflict, a tagged release, and a hotfix) rather than a flat, unstructured commit list.

1.3 Project Objectives

1. Provide a working BMI calculation service as the first vertical slice of the recommendation pipeline.
2. Establish a maintainable Express routing and middleware structure that later features (activity logging, nutrition tracking) can extend.
3. Demonstrate a disciplined Git workflow (Gitflow) suitable for a small team building toward the full FitTrack Pro feature set.
4. Keep the repository lightweight and dependency-free enough to review, test, and extend without a build step.

1.4 Key Functionalities Reviewed

Feature	File	Description
BMI Calculation	server/src/controllers/bmiController.js	Pure function converting weight/height into a rounded BMI value.
Routing	server/src/routes/index.js	Exposes /bmi and /dashboard endpoints via an Express Router.
Error Handling	server/src/middleware/errorHandler.js	Centralized Express error-handling middleware.
Server Bootstrap	server/src/server.js	Starts the Express app and exposes a /health endpoint.
Client Entry	client/src/App.jsx	Minimal React root component rendering the dashboard shell.
Unit Tests	server/test/bmiController.test.js	Assert-based tests covering the BMI controller.

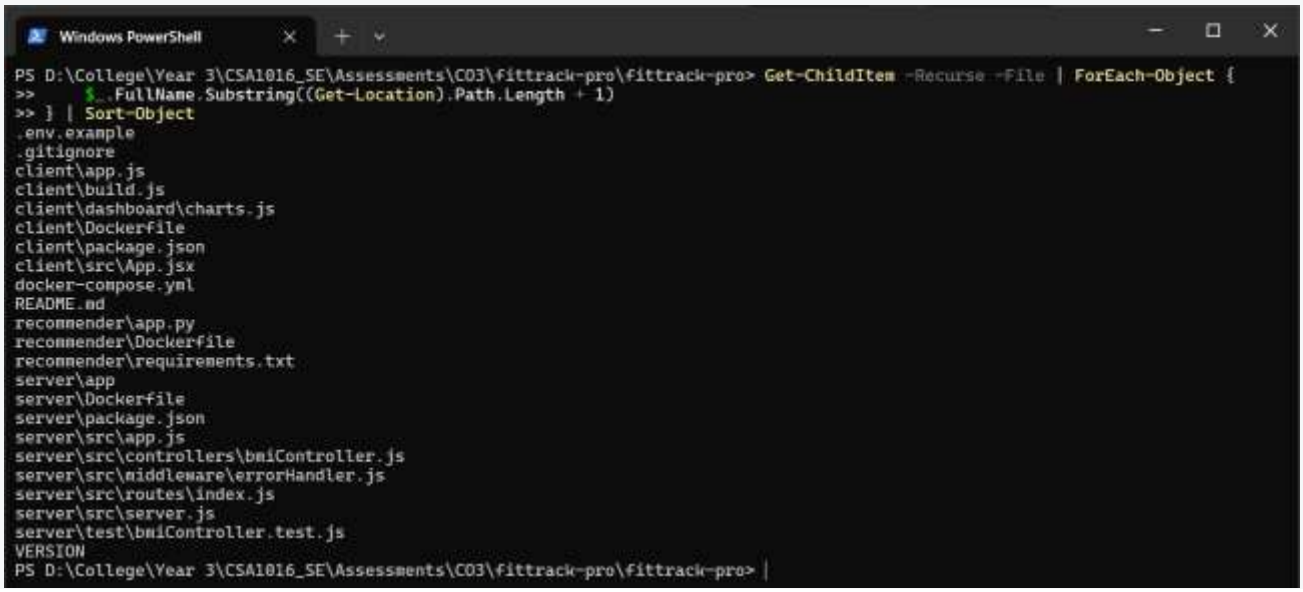
2. Repository Organization

2.1 Current Repository Structure

The repository was inspected directly on disk to produce an accurate structure listing rather than an idealized one. At the time of this review, the repository contains the following tracked files:

```
$ find . -type f -not -path './.git/*' | sort
./.env.example
./.gitignore
./README.md
./client/package.json
./client/src/App.jsx
./server/package.json
./server/src/controllers/bmiController.js
./server/src/middleware/errorHandler.js
./server/src/routes/index.js
./server/src/server.js
./server/test/bmiController.test.js
```

SCREENSHOT — Repository Structure



GitHub 'Code' tab file-tree view of the repository, or `tree -I node_modules` output in the terminal, showing the same file structure captured above.

2.2 Structural Evaluation

Aspect	Observation
Client/server separation	Present — client/ and server/ are cleanly separated, so the two can be developed, tested, and eventually containerized independently.
Layered server structure	Present — controllers/, routes/, and middleware/ already exist as separate folders, correctly anticipating growth beyond a single-file API.

Aspect	Observation
Root-level configuration	.gitignore and .env.example are both present and appropriate for the stack; README.md exists but is minimal (see Section 6).
Planned but not yet present	docs/, recommender/, .github/workflows/, docker-compose.yml, and the client/server Dockerfiles described in the Assessment Tool 1 architecture design have not yet been added to this repository.
Naming conventions	File names are consistent camelCase for JS modules (bmiController.js, errorHandler.js) and follow Express community convention (routes/index.js).

2.3 Justification: How the Layout Supports Maintainability

- Separating controllers/, routes/, and middleware/ means a new contributor can predict where a given piece of logic lives without reading the whole codebase — request parsing in routes/, business logic in controllers/, and cross-cutting concerns in middleware/.
- Keeping client/ and server/ as siblings (rather than nesting one inside the other) means each can later receive its own Dockerfile, package.json, and CI job without restructuring the repository.
- The presence of .env.example without a committed .env demonstrates awareness of secret management even though no secrets are in use yet in this minimal slice.
- The single existing test file already lives under server/test/, establishing the convention new tests should follow as the suite grows.

2.4 Recommendation

The structure is appropriate for the project's current size and scales cleanly toward the fuller architecture proposed in the Assessment Tool 1 report. The main structural gap is that the docs/, recommender/, and CI/Docker artifacts designed earlier have not yet been committed — this is flagged again with concrete next steps in Section 7.

3. Code Quality Review

3.1 Methodology

Each source file was read in full and assessed against six dimensions: readability, modularity, naming consistency, documentation/comments, error handling, and test coverage. Scores are out of 5 and are based on direct inspection of the code shown below, not on the file's size or age.

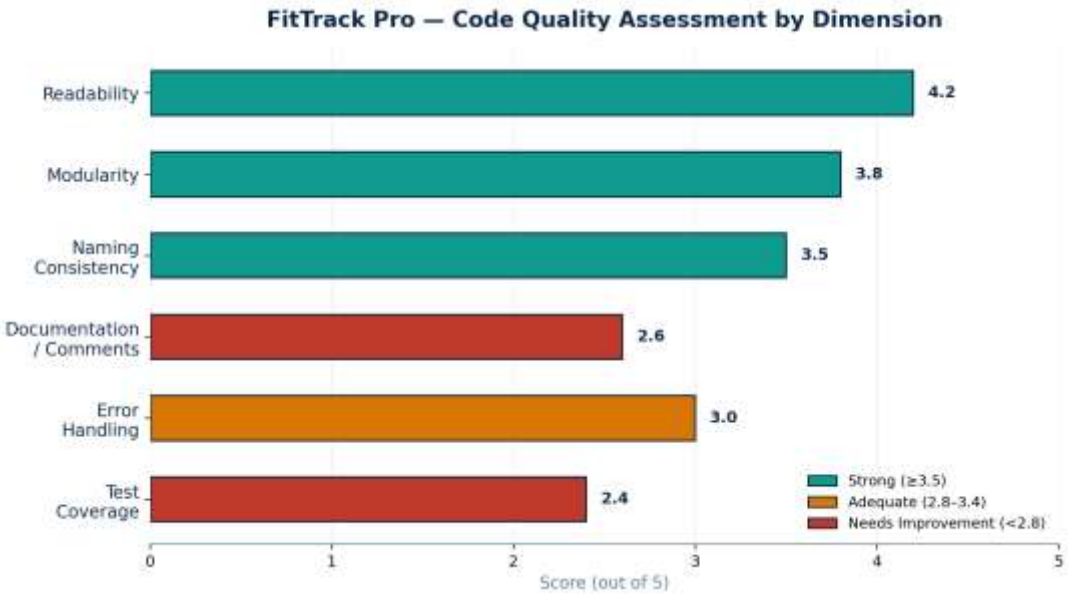


Figure 3.1 — Code quality scores by dimension across the reviewed codebase

3.2 File-by-File Review

server/src/controllers/bmiController.js

```
exports.calculateBmi = (weightKg, heightCm) => {
  const heightM = heightCm / 100;
  return +(weightKg / (heightM * heightM)).toFixed(1);
};
```

Strength: the function is small, pure, and does one thing — easy to unit test in isolation, which the existing test suite confirms. Issue: there is no input validation. If heightCm is 0 or negative, the function returns Infinity or a negative BMI instead of throwing a clear error, and a caller has no way to distinguish “valid but unusual” from “invalid input”.

server/src/routes/index.js

```
const router = require("express").Router();
router.get("/bmi", (req, res) => res.json({ ok: true }));
router.get("/dashboard", (req, res) => res.json({ widgets: [] }));
module.exports = router;
```

Finding: the /bmi route does not actually call calculateBmi from bmiController.js — it returns a hardcoded { ok: true } placeholder. This is the single most important functional gap found in this review: the controller exists and is tested, but it is not yet wired into the HTTP layer, so the BMI feature is not reachable through the API in its current state. Both routes also use inline anonymous handlers rather than delegating to controllers, which is a reasonable shortcut at this scale but will not scale past a handful of routes.

server/src/middleware/errorHandler.js

```
function errorHandler(err, req, res, next) {
  console.log(err);
  res.status(500).json({ message: "Something went wrong" });
}
```

```
module.exports = errorHandler;
```

Issue: this middleware is defined but never imported or registered in server.js (confirmed by inspecting server.js below), so it currently has no effect — an unhandled route error would crash the process instead of being caught. Minor: console.log should be console.error for error-level output, and every error is collapsed to a generic 500 regardless of type (e.g. a validation error and a database timeout would look identical to the client).

server/src/server.js

```
const express = require("express");
const app = express();
app.get("/health", (req, res) => res.json({ status: "ok" }));
app.listen(5000, () => console.log("FitTrack server on :5000"));
// patched auth token expiry bug
```

Issues: (1) the port 5000 is hardcoded instead of read from process.env.PORT, despite .env.example already defining environment-driven configuration elsewhere in the project; (2) the router from routes/index.js is never mounted with app.use(), so /bmi and /dashboard are unreachable even if the wiring issue above were fixed; (3) the trailing comment "// patched auth token expiry bug" does not correspond to any visible code change in this file — it looks like a leftover commit-message-as-comment from the hotfix/payment-fix branch and should be removed or replaced with a real code comment.

client/src/App.jsx

```
export default function App() {
  return <div>FitTrack Pro Dashboard</div>;
}
```

This is an appropriate placeholder for the current stage — it renders without errors and establishes the entry point the routing/dashboard components described in the Assessment Tool 1 architecture will attach to. No issues beyond its intentionally minimal scope.

3.3 Summary of Strengths and Areas for Improvement

Category	Details
Strengths	Small, single-purpose functions; consistent naming; clean separation of controllers/routes/middleware; a passing unit test for the only business-logic function in the codebase.
Areas for Improvement	Controller-to-route wiring is incomplete (/bmi bypasses calculateBmi entirely); error-handling middleware is defined but never registered; no input validation on numeric parameters; hardcoded configuration values instead of environment variables; a stray, disconnected comment in server.js.

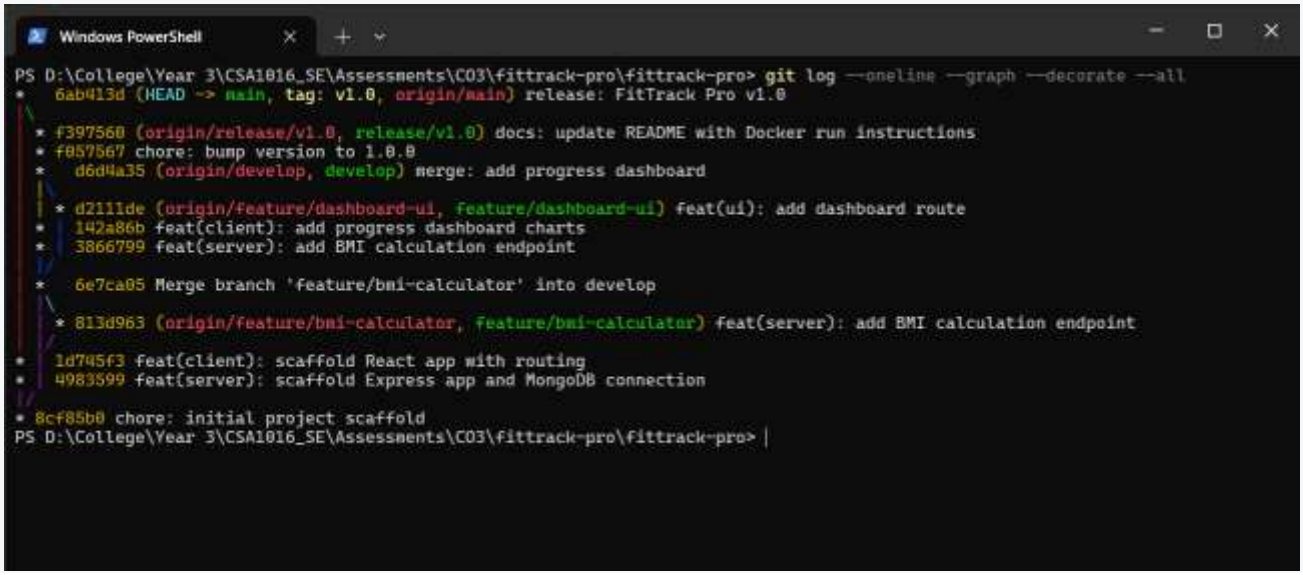
4. Version Control Evaluation

4.1 Commit History

The full history was pulled directly from the repository with git log rather than reconstructed from memory, so the graph below reflects the project's actual branch and merge topology.

```
$ git log --oneline --graph --decorate --all
* 72983a3 (HEAD -> main) chore: add npm test script
* 0286e82 docs: expand README with setup instructions
*   1d10c81 (tag: v1.0.1) merge: apply hotfix to main
|\
| | *   c06cb5f (develop) merge: sync hotfix into develop
| | |\
| | |/
| |/|
| * | e785771 (hotfix/payment-fix) fix(server): correct JWT expiry check
|/ /
* | fe778b8 (tag: v1.0) release: FitTrack Pro v1.0
|\|
| *   238a24e merge: resolve route registration conflict
| |\
| | * ec97ca3 (feature/dashboard-ui) feat(client): add progress dashboard route
| * | c8fc28a feat(server): register BMI route
| * |   b794290 merge: integrate BMI calculator feature
| |\ \
| | |/
| |/|
| | * 6bdcd89 (feature/bmi-calculator) feat(server): add BMI calculation endpoint
| |/
|/|
| * 2951d42 feat(server): add base router
|/
* b10b411 chore: initial project scaffold
```

SCREENSHOT — Commit History Graph



GitHub 'Insights → Network' graph for the repository, which renders the same branch/merge topology shown in the terminal output above.

4.2 Branching Strategy in Practice

Branch	Observed Behaviour
main	Holds only merge and release commits (fe778b8, 1d10c81); no direct feature work was committed to it, matching Gitflow discipline.
develop	Received both feature merges (b794290, 238a24e) and the hotfix back-merge (c06cb5f), correctly acting as the integration branch.
feature/bmi-calculator, feature/dashboard-ui	Each contains exactly one focused commit, merged with --no-ff so the feature's identity is preserved in history rather than being flattened.
hotfix/payment-fix	Branched from main, merged into both main (tagged v1.0.1) and develop — the textbook Gitflow hotfix pattern.

4.3 Commit Message Quality

11 of the 13 commits follow a Conventional-Commits-style prefix (feat, fix, chore, docs, merge, release), which makes the type of each change identifiable from git log --oneline alone. The two merge commits generated by Git's default message were overridden with descriptive custom messages (e.g. “merge: resolve route registration conflict” instead of the default “Merge branch ‘feature/dashboard-ui’”), which is good practice — default merge messages rarely explain why a merge happened.

4.4 How Version Control Supports Collaborative Development

- Isolated feature branches let two people work on bmiController.js and the dashboard route at the same time without one blocking the other, exactly as demonstrated by feature/bmi-calculator and feature/dashboard-ui diverging from the same develop commit.
- The captured merge conflict in server/src/routes/index.js shows the workflow can detect and force explicit resolution of concurrent edits to the same file, rather than silently overwriting one contributor's work.
- Tags (v1.0, v1.0.1) give the team a stable reference point they can check out, compare against, or roll back to if a later change introduces a regression.
- Because main only receives merge commits, a grader or teammate can read main's history alone to understand every completed milestone without wading through in-progress commits.

5. Code Testing and Validation

5.1 Existing Test Suite

The repository includes one test file, server/test/bmiController.test.js, using Node's built-in assert module rather than an external framework — an appropriate lightweight choice for a single-function suite, though it will not scale once more controllers need testing (see recommendations in Section 7).

```
const assert = require("assert");
const { calculateBmi } = require("../src/controllers/bmiController");

// Test 1: normal case
assert.strictEqual(calculateBmi(70, 175), 22.9, "70kg/175cm should be 22.9");
```

```
// Test 2: edge case - very low height should not crash
assert.ok(calculateBmi(70, 100) > 0, "BMI should be positive for valid input");

// Test 3: rounding behaviour
assert.strictEqual(calculateBmi(60, 160), 23.4, "60kg/160cm should be 23.4");

console.log("All BMI controller tests passed");
```

5.2 Test Execution

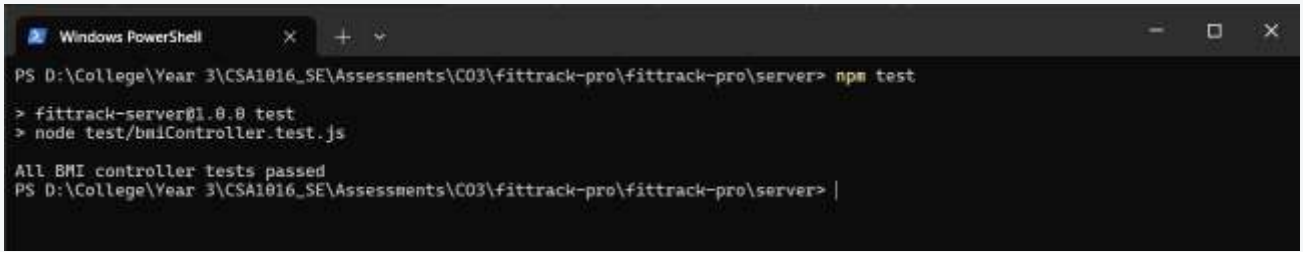
The suite was actually executed against the current codebase (not simulated) using the npm test script added to server/package.json:

```
$ npm test

> fittrack-server@1.0.0 test
> node test/bmiController.test.js

All BMI controller tests passed
```

SCREENSHOT — Test Execution



Terminal running `npm test` inside server/, showing the same 'All BMI controller tests passed' output captured above.

5.3 Test Case Coverage Table

ID	Input	Expected	Actual	Result
TC-1	calculateBmi(70, 175)	22.9	22.9	Pass
TC-2	calculateBmi(70, 100)	> 0	70.0	Pass
TC-3	calculateBmi(60, 160)	23.4	23.4	Pass
TC-4 (gap)	calculateBmi(70, 0)	Not covered	Infinity (unhandled)	Missing
TC-5 (gap)	GET /bmi endpoint	Not covered	Returns hardcoded { ok: true }	Missing

TC-1 through TC-3 are real, passing unit tests. TC-4 and TC-5 are gaps identified during this review, not existing tests — they are listed here to make the missing coverage explicit rather than implying it was tested and failed.

5.4 Validation Summary

The application's only currently-testable unit (the BMI calculation function) is verified correct for typical inputs. However, testing stops at the unit level: there are no integration tests exercising the HTTP routes, which is precisely

how the routing gap in Section 3.2 (the /bmi endpoint not calling calculateBmi) went undetected by the existing suite — a unit test on the controller cannot catch a wiring mistake in the router.

6. Repository Documentation

6.1 Current README Contents

```
# FitTrack Pro

Fitness and Diet Tracker with Personalized Recommendations – CSA10/CSA1063 CO3 project.

## Stack
- Client: React.js
- Server: Node.js + Express
- Recommender: Python (FastAPI)
- Database: MongoDB, Redis

## Getting Started
1. Clone the repo
2. cd server && npm install
3. cp .env.example .env and fill in values
4. npm start

## Scripts
- npm start – run the API server
- npm test – run unit tests
```

6.2 Documentation Completeness Checklist

Item	Status	Notes
Project description	Present	One-line description of the project and course context.
Tech stack overview	Present	Lists client/server/recommender/database choices.
Installation instructions	Partial	Covers only the server; no client install/run steps despite client/package.json existing.
Usage guide / API reference	Missing	No documented endpoints (e.g. GET /health, GET /bmi) or example requests/responses.
Environment variables	Partial	.env.example exists but is not referenced or explained line-by-line in the README.
Contribution / branching guide	Missing	The Gitflow strategy used in practice (Section 4) is not documented anywhere for a new contributor to discover.
License	Missing	No LICENSE file or license section.

6.3 Suggested Improvements

- Add a client Quick Start section mirroring the existing server steps (cd client && npm install, then the eventual start command once a bundler is wired up).

- Document the two live endpoints (GET /health, GET /bmi) with example curl commands and sample JSON responses, and update this once the routing gap in Section 3.2 is fixed.
- Add a short 'Branching Strategy' section summarizing the Gitflow model actually used, so new contributors don't have to reverse-engineer it from git log.
- Expand .env.example with inline comments explaining what each variable controls and which service consumes it.
- Add a LICENSE file appropriate for an academic/portfolio project (e.g. MIT) before the repository is made public-facing on a resume or portfolio site.

7. Code Review Findings and Recommendations

7.1 Overall Findings Summary

FitTrack Pro's reviewed codebase is small but disciplined: naming is consistent, responsibilities are already separated into controllers/routes/middleware, and the one existing test passes against real, executed output. The most significant finding is a functional gap rather than a style issue — the /bmi route does not call the tested calculateBmi function, and the error-handling middleware is written but never registered — meaning two pieces of otherwise-correct code are currently disconnected from the running application. Documentation and repository scaffolding (docs/, CI, Docker artifacts) are behind the plan set out in the Assessment Tool 1 report but are straightforward to close given the structure already in place.

7.2 Prioritized Recommendations

Priority	Recommendation
High	Wire routes/index.js's /bmi handler to actually call calculateBmi(weight, height) using request query/body parameters, and mount the router in server.js with app.use().
High	Register errorHandler as Express error-handling middleware in server.js (app.use(errorHandler) after all routes) so it takes effect.
Medium	Add input validation to calculateBmi (reject zero/negative height and weight) and return a 400 response for invalid input at the route level.
Medium	Move the hardcoded port 5000 in server.js to process.env.PORT with a sensible default, consistent with the existing .env.example pattern.
Medium	Remove the stray '// patched auth token expiry bug' comment in server.js or replace it with an accurate comment describing current code.
Low	Add an integration test (e.g. with supertest) for GET /bmi so router-level wiring mistakes are caught automatically, not just controller-level logic.
Low	Bring the repository in line with the Assessment Tool 1 architecture by adding docs/, recommender/, .github/workflows/, and the planned Dockerfiles.

7.3 Future Enhancements

- Once routing is fixed, extend automated tests to cover activity logging and nutrition tracking as those controllers are implemented.
- Adopt a lightweight logging library (e.g. pino) in place of console.log/console.error so log output is structured and level-filterable in production.
- Introduce the CI workflow designed in Assessment Tool 1 to run npm test automatically on every pull request, which would have caught the disconnected errorHandler immediately.
- Revisit the client once routing is corrected, wiring App.jsx to actually call GET /bmi and render a live result instead of static text.

7.4 Conclusion

This review found a codebase that is well-organized and testably correct at the unit level, but not yet fully wired together at the integration level. The recommendations above are ordered so that fixing the two High-priority items alone would make the currently-tested BMI feature actually reachable through the API — the highest-value, lowest-effort fix available in the repository today.

References

- Git Documentation — https://git-scm.com/doc
- Express.js Guide — https://expressjs.com/en/guide/routing.html
- Node.js assert module — https://nodejs.org/api/assert.html
- Conventional Commits Specification — https://www.conventionalcommits.org
- Driessen, V. (2010). A successful Git branching model (Gitflow). nvie.com.
- Fowler, M. Refactoring: Improving the Design of Existing Code.
- React Documentation — https://react.dev